

LEZIONE 4 – CLASSI DI PROGETTO E DIAGRAMMI DI INTERAZIONE

Questa lezione conclude la parte di progettazione del sistema di rubrica telefonica, presentando classi di progetto e diagrammi di interazione (diagrammi di sequenza). Di seguito viene riportata la descrizione del sistema Rubrica:

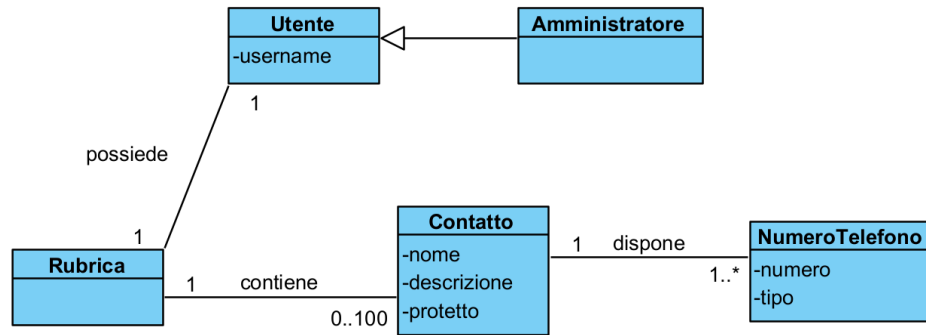
“Un utente del sistema – identificato da uno username – può gestire una rubrica telefonica, che include una lista di contatti (massimo 100). Ogni contatto è caratterizzato da un nome univoco, una descrizione, e uno o più numeri di telefono (fisso casa, ufficio, cellulare, etc). L'utente può inserire nuovi contatti nella rubrica telefonica, rimuovere quelli presenti, o aggiornarne i dati. I contatti possono essere aggiunti in modalità “protetta”, in modo da impedire la rimozione dalla rubrica. Il sistema deve anche consentire la ricerca di un contatto presente in rubrica, inserendo il nome del contatto oppure un numero di telefono. Il sistema prevede anche un utente amministratore che, oltre alle attività precedenti, può monitorare le attività svolte dagli altri utenti.”

Progettazione guidata da operazioni di sistema

Nella lezione precedente abbiamo progettato l'architettura logica del sistema. Ora possiamo procedere con la realizzazione dei casi d'uso. Per realizzare un caso d'uso è necessario modellare le operazioni di sistema, individuate a partire dagli SSD e descritte per mezzo dei contratti, in termini di diagrammi di interazione (nel nostro caso, diagrammi di sequenza). Ci concentreremo sul caso d'uso **InserisciContatto**, già trattato precedentemente, limitatamente all'operazione di sistema **inserisciNuovoContatto** descritta dal contratto seguente:

Contratto inserisciNuovoContatto
Operazione: inserisciNuovoContatto()
Riferimenti caso d'uso: InserisciContatto
Precondizioni: è in corso l'aggiunta in Rubrica <i>r</i> di un nuovo contatto
Postcondizioni:
- È stata creata un'istanza <i>c</i> di Contatto (<u>creazione di oggetto</u>)
- Gli attributi di <i>c</i> sono stati inizializzati (<u>modifica di attributi</u>)
- l'istanza <i>c</i> di Contatto è stata associata alla Rubrica <i>r</i> (<u>formazione di collegamento</u>)

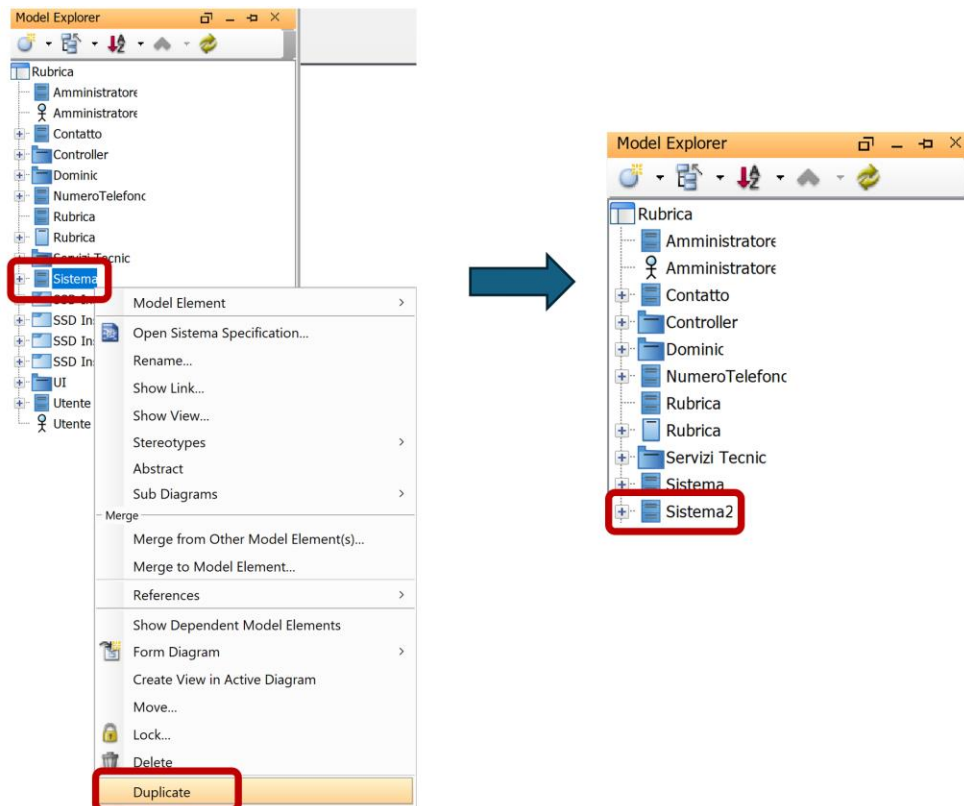
Le operazioni di sistema individuate negli SSD, a livello software determinano chiamate a operazioni che passano dallo strato UI agli strati sottostanti; in particolare, l'architettura logica introdotta nella lezione precedente prevede strati UI, Controller, Dominio e Servizi Tecnici. Mentre modelleremo l'operazione di sistema **inserisciNuovoContatto** in termini di diagramma di sequenza, in parallelo produrremo il corrispettivo diagramma delle classi di progetto, che rifletterà le entità software coinvolte. Per definire le classi software prenderemo ispirazione anche dal modello di dominio prodotto in precedenza, di seguito riportato.



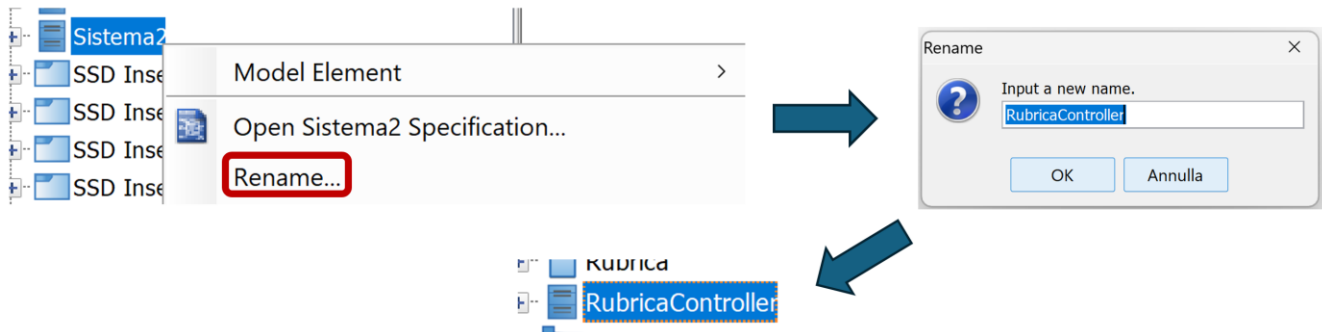
Modellazione operazione di sistema ‘inserisciNuovoContatto’

Nel modellare l’operazione di sistema **inserisciNuovoContatto** ci limiteremo a osservare gli aspetti che coinvolgono gli strati Controller e Dominio, tendenzialmente i più interessanti. Come punto di partenza, introduciamo nel modello una classe che rappresenterà il punto di accesso al nostro sistema, assegnandole la responsabilità di mediare tra gli eventi di sistema generati dallo strato UI e le classi sottostanti appartenenti allo strato di Dominio.

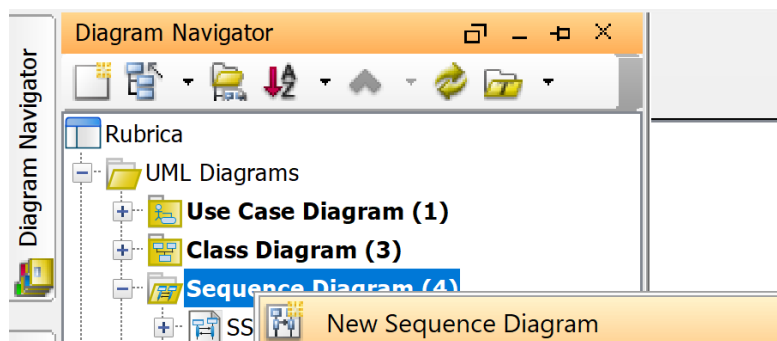
Nella scorsa lezione abbiamo introdotto una classe “Sistema” utile a raccogliere tutte le operazioni di sistema usate negli SSD. Possiamo utilizzare una variante software di questa classe per rappresentare il punto di accesso al nostro sistema. Aprendo il progetto **Rubrica**, dal pannello **Model Explorer** sulla sinistra, clicchiamo col tasto destro sulla classe “Sistema” in elenco, quindi selezioniamo la voce **Duplicate**. L’operazione produrrà una copia esatta della classe “Sistema”, denominata “Sistema2”.



Cliccando su di essa col tasto destro del mouse, selezioniamo la voce **Rename** e rinominiamola “RubricaController”.



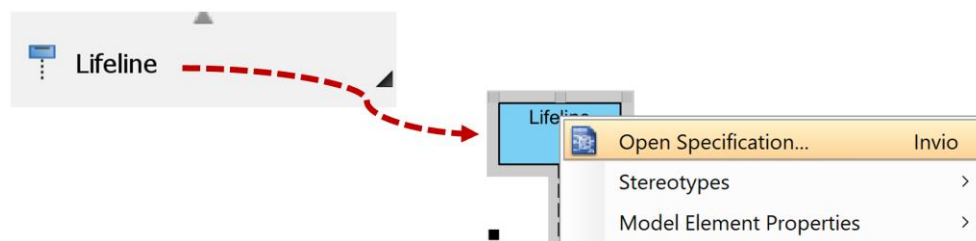
Creiamo ora un diagramma di sequenza per modellare l’operazione di sistema *InserisciNuovoContatto* (da **Diagram Navigator**, tasto destro su **Sequence Diagram** > **New Sequence Diagram**).



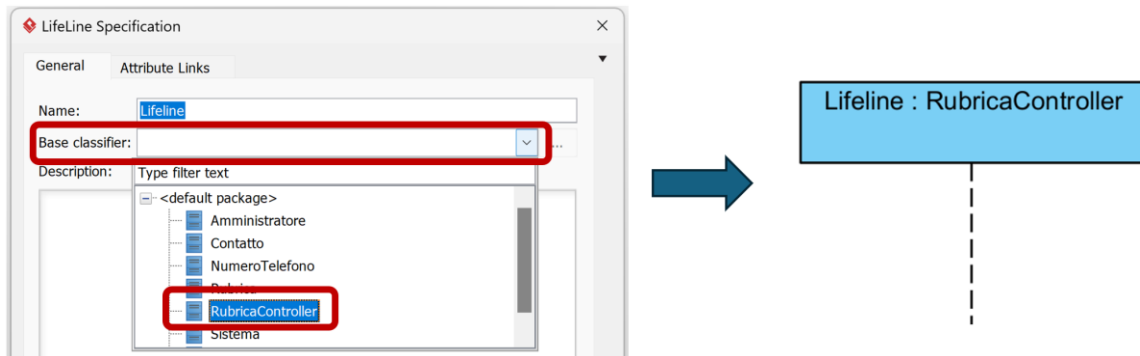
Rinominiamolo come “SD InserisciNuovoContatto”, facendo doppio click sul nome. È importante dare un nome significativo al diagramma di sequenza, e soprattutto distinguere i diagrammi di sequenza (SD) dai diagrammi di sequenza di sistema (SSD) presenti. Otterremo il seguente risultato.



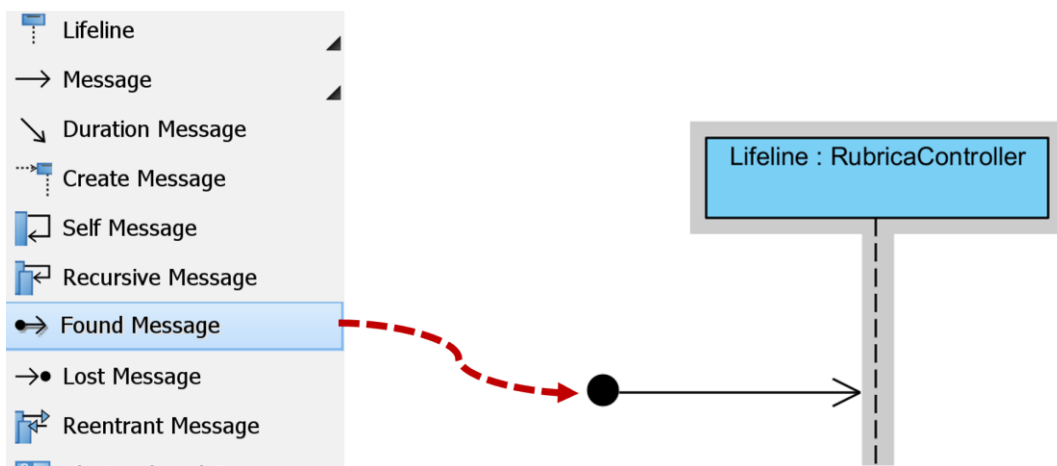
Ora possiamo introdurre una lifeline relativa al nostro punto di accesso al sistema nel diagramma di sequenza appena creato. Selezioniamo dalla **Palette** a sinistra la voce **Lifeline**, trasciniamola nel diagramma; quindi, clicchiamo su di essa col tasto destro del mouse e selezioniamo la voce **Open Specification**.



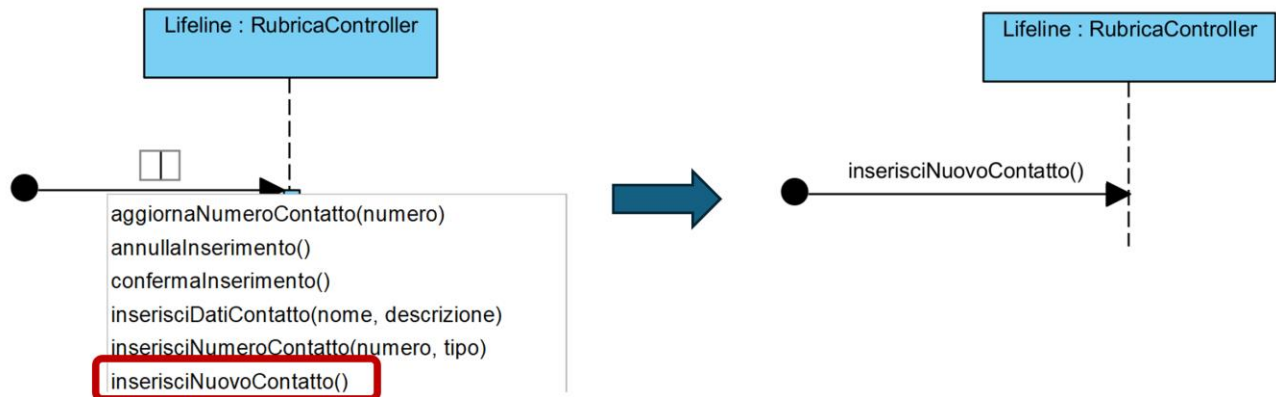
Nella finestra che si aprirà, espandiamo le voci sotto **Base classifier**, selezioniamo “RubricaController” e confermiamo. Ciò collegherà la lifeline alla classe precedentemente creata, permettendo di far riferimento alle sue operazioni durante la modellazione del diagramma di sequenza. Il risultato, trascinando la lifeline in modo che il testo sia interamente visibile, è il seguente.



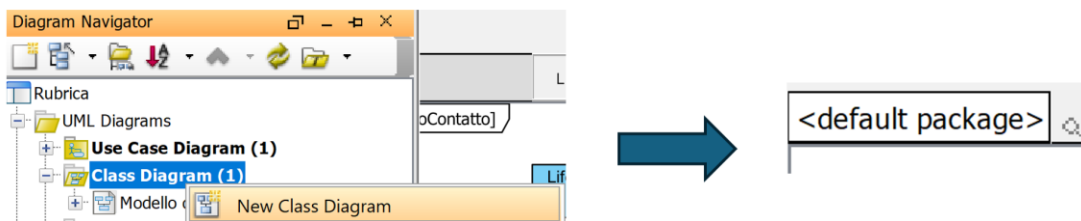
Dato che “RubricaController” fa da mediatore tra lo strato UI – non trattato nel diagramma – e lo strato di Dominio sottostante, per rappresentare un evento di sistema esterno diretto a “RubricaController” introduciamo un **messaggio trovato** (anche chiamato **found message**). Clicchiamo sulla voce **Found Message** nella **Palette**, quindi trasciniamo a partire da uno spazio vuoto del diagramma fino a toccare la lifeline, come mostrato di seguito.



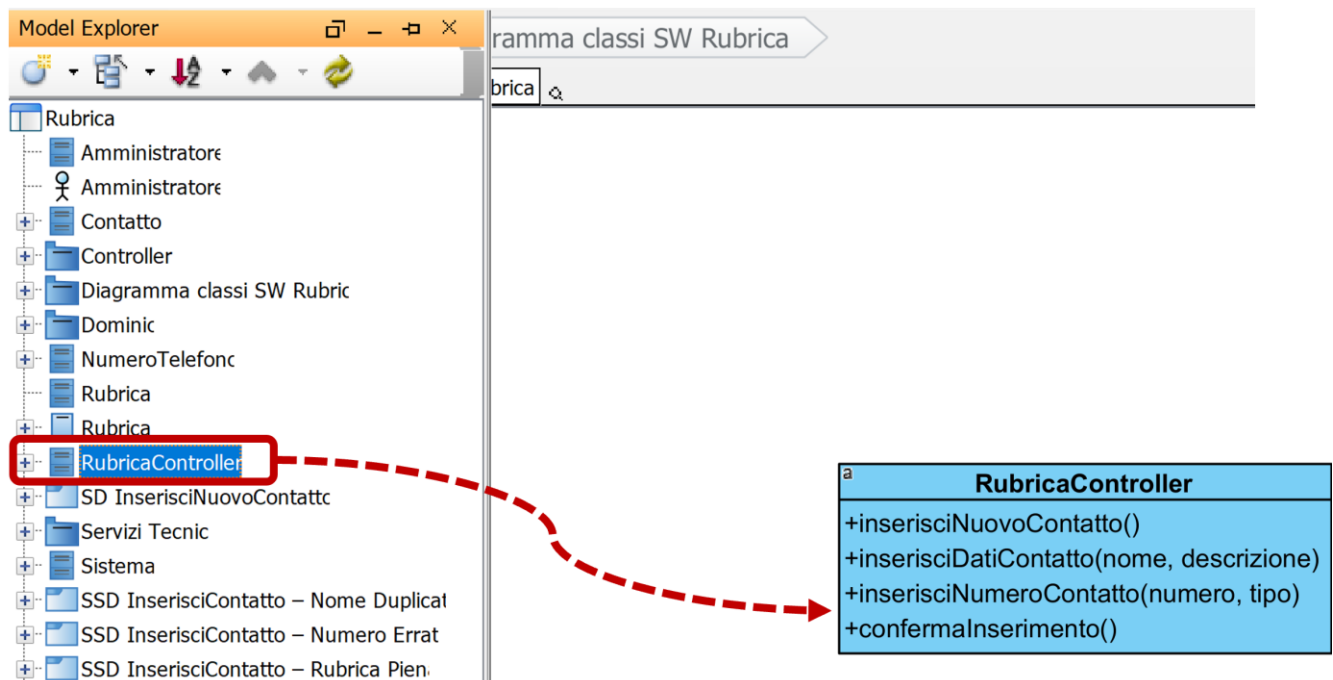
Come nel caso di messaggi “classici”, Visual Paradigm propone una lista di voci selezionabili per associare il messaggio a un’operazione esistente e relativa alla classe: selezioniamo la voce `inserisciNuovoContatto()`. Il **messaggio trovato**, che giunge dallo strato superiore UI a seguito della richiesta dell’utente, invocherà l’omonima operazione di sistema incapsulata in “RubricaController”, attivando così tutto il diagramma.



Prima di procedere oltre nel diagramma di sequenza, creiamo un diagramma delle classi al fine di contenere le classi SW coinvolte. Dal pannello **Diagram Navigator** a sinistra, con tasto destro su **Class Diagram** selezioniamo la prima voce **New Class Diagram**. Rinominiamo "<default package>" in alto in "Diagramma classi SW Rubrica" (non stiamo dando un nome solo al diagramma ma al package che lo conterrà, vedremo tra poco perché).

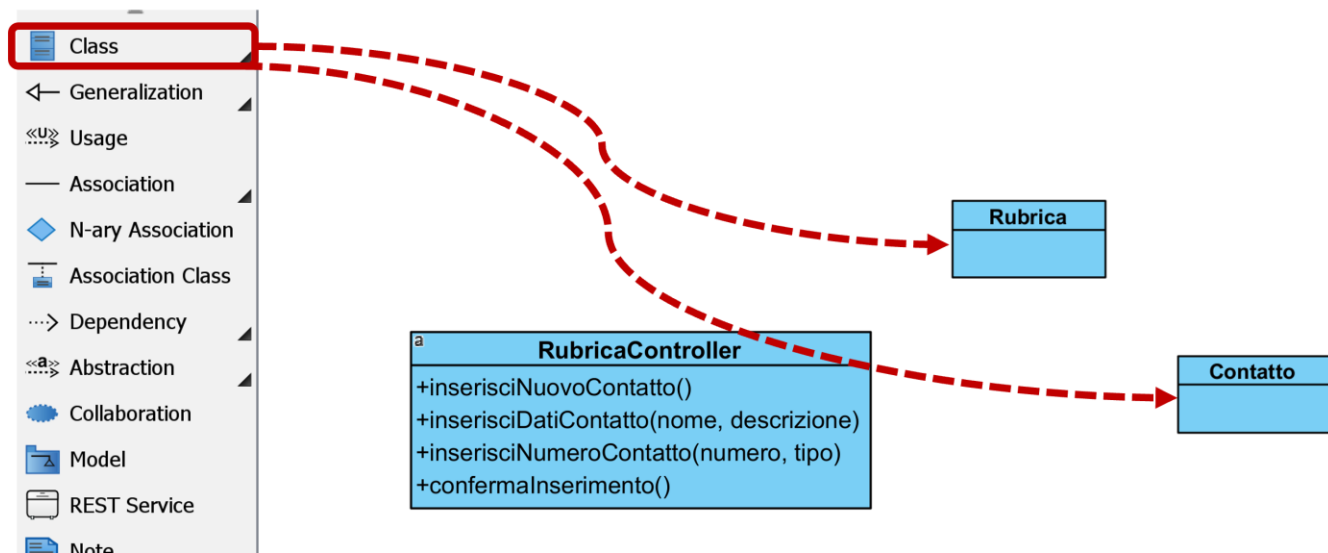


Dal pannello **Model Explorer** a sinistra, trasciniamo la classe "RubricaController" nel diagramma delle classi appena creato. Si ricorda che il **Model Explorer** gestisce il modello UML relativo a tutto il progetto; pertanto, è uno strumento molto utile per tracciare e riusare i costrutti UML introdotti in precedenza.

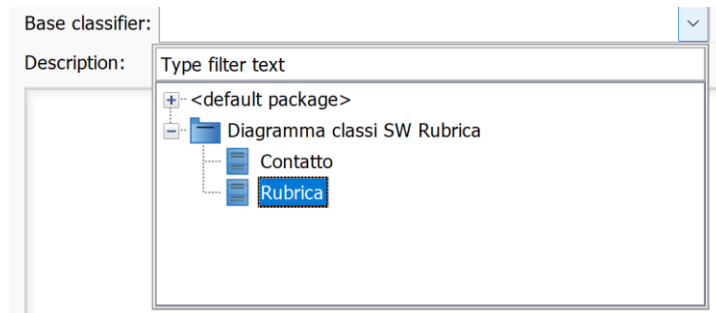


A questo punto, per proseguire la modellazione nel diagramma di sequenza, bisogna ragionare su come “RubricaController” possa gestire l’operazione di sistema `inserisciNuovoContatto()`. Rileggendo il contratto dell’operazione di sistema, come riportato a inizio documento, scopriamo che al suo completamento viene creata un’istanza della classe “Contatto” (post-condizioni “È stata creata un’istanza *c* di Contatto” e “Gli attributi di *c* sono stati inizializzati”). Osservando invece il modello di dominio possiamo vedere che esiste un’associazione tra “Contatto” e “Rubrica”. Poiché una rubrica per sua natura contiene e gestisce molti contatti, possiamo ragionevolmente pensare che “Rubrica” sia responsabile della creazione di un contatto.

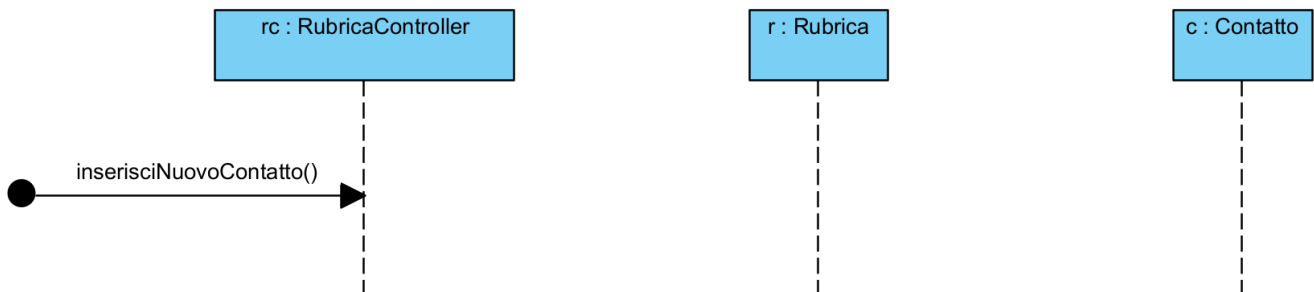
Introduciamo quindi due nuove classi nel diagramma delle classi di progetto, chiamate “Rubrica” e “Contatto”. Si noti che queste classi, pur avendo lo stesso nome delle classi nel modello di dominio, rappresentano concetti software differenti.



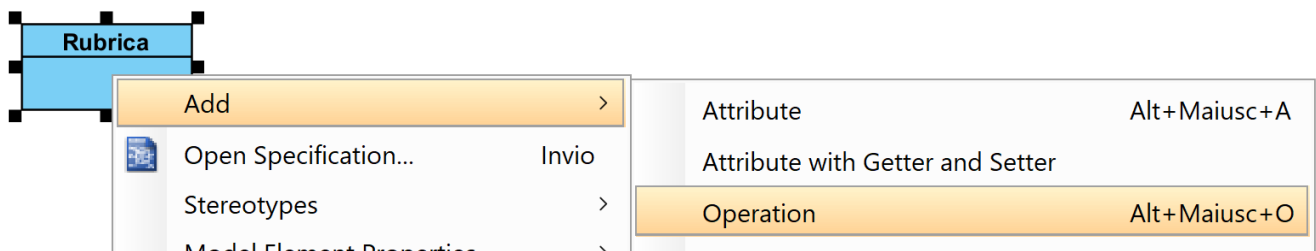
Torniamo ora sul diagramma di sequenza (il pannello **Diagram Navigator**, come suggerisce il nome, ci viene molto utile per navigare tra diagrammi quando il modello inizia a complicarsi) e aggiungiamo due lifeline, una per “Rubrica” e una per “Contatto”. Come fatto per “RubricaController”, via tasto destro > **Open Specification**, selezioniamo un **Base classifier** per ciascuna classe, espandendo le voci relative (una lifeline andrà associata alla classe “Rubrica” e una alla classe “Contatto”). Si noti che avendo dato un nome al package contenente il diagramma delle classi SW, anche il **Base classifier** è cambiato, permettendoci di disambiguare dai riferimenti alle classi del modello di dominio aventi lo stesso nome delle classi SW (es. Rubrica come concetto del dominio è diversa da Rubrica come classe SW). Un’alternativa più semplice all’assegnazione dei nomi ai package, ma non particolarmente elegante, è quella di utilizzare nomi diversi per ogni classe (es. RubricaModello e RubricaSW).



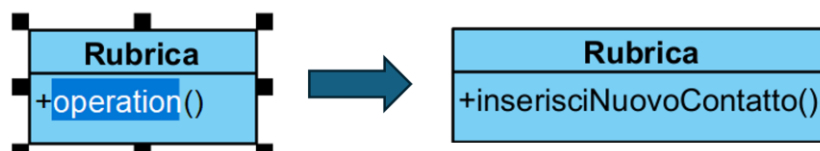
Rinominiamo le lifeline aggiunte in “rc” per “RubricaController”, “r” per “Rubrica” e “c” per “Contatto”, ottenendo quanto di seguito.



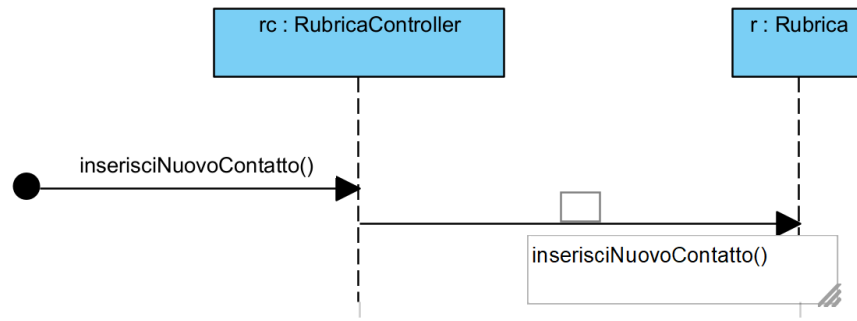
Ora vogliamo aggiungere il messaggio `inserisciNuovoContatto()` da “RubricaController” a “Rubrica”, in modo da assegnare la responsabilità di creare contatti a “Rubrica”. Come prima cosa, dobbiamo inserire un’operazione nella classe “Rubrica”, quindi via **Diagram Navigator** torniamo sul diagramma delle classi SW, selezioniamo la classe “Rubrica” e, cliccando col tasto destro su di essa, selezioniamo **Add > Operation**.



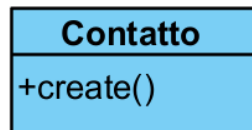
Ciò aggiungerà un’operazione senza nome associata alla classe. Diamo il nome `inserisciNuovoContatto`.



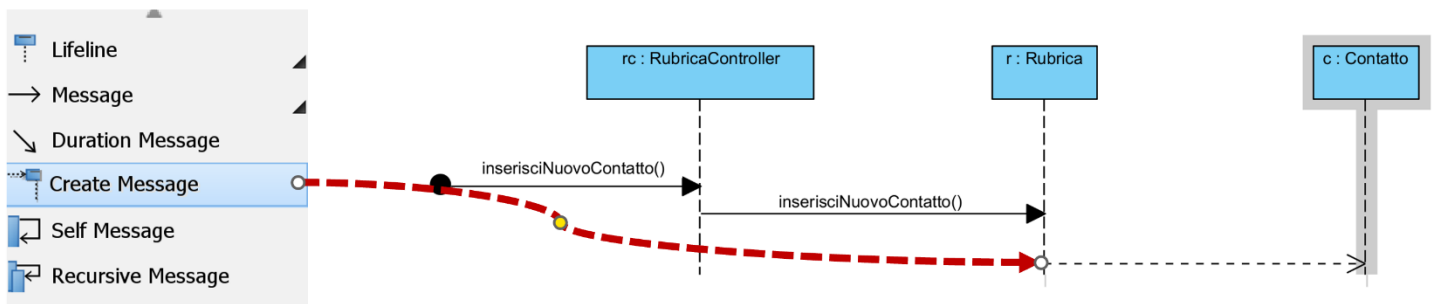
A questo punto possiamo inviare un messaggio da “RubricaController” a “Rubrica”, passando nuovamente al diagramma di sequenza, selezionando l’operazione appena aggiunta.



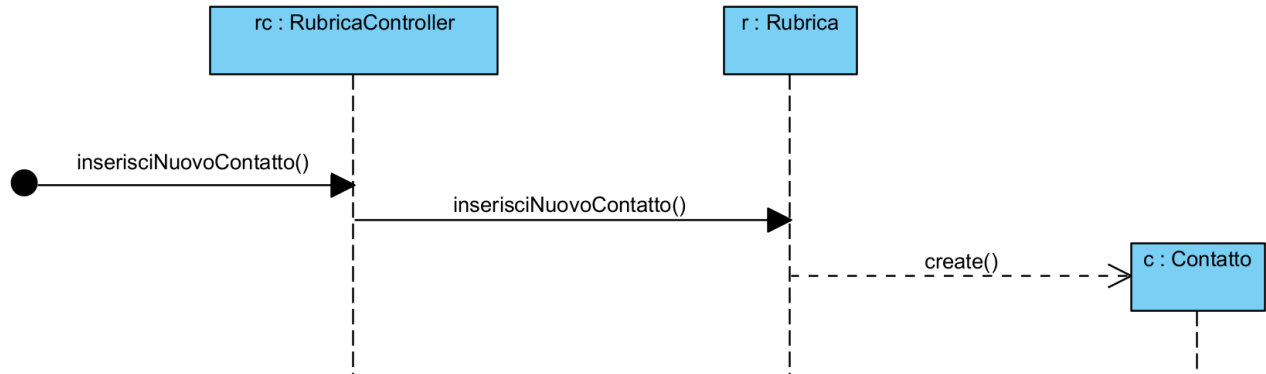
Vogliamo ora aggiungere il **messaggio di creazione** che partirà dalla lifeline “Rubrica” alla lifeline “Contatto”, dato che vogliamo rappresentare la creazione di un nuovo contatto, la cui responsabilità è in mano alla classe “Rubrica”. Il primo passo avviene nuovamente attraverso il diagramma delle classi, dove creeremo un’operazione create() per la classe “Contatto”, come visto in precedenza.



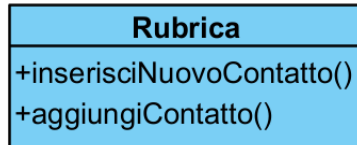
Tornando sul diagramma di sequenza, clicchiamo l’icona **Create Message** dalla **Palette**, quindi trasciniamo da “Rubrica” a “Contatto”, selezionando l’operazione create() che verrà proposta.



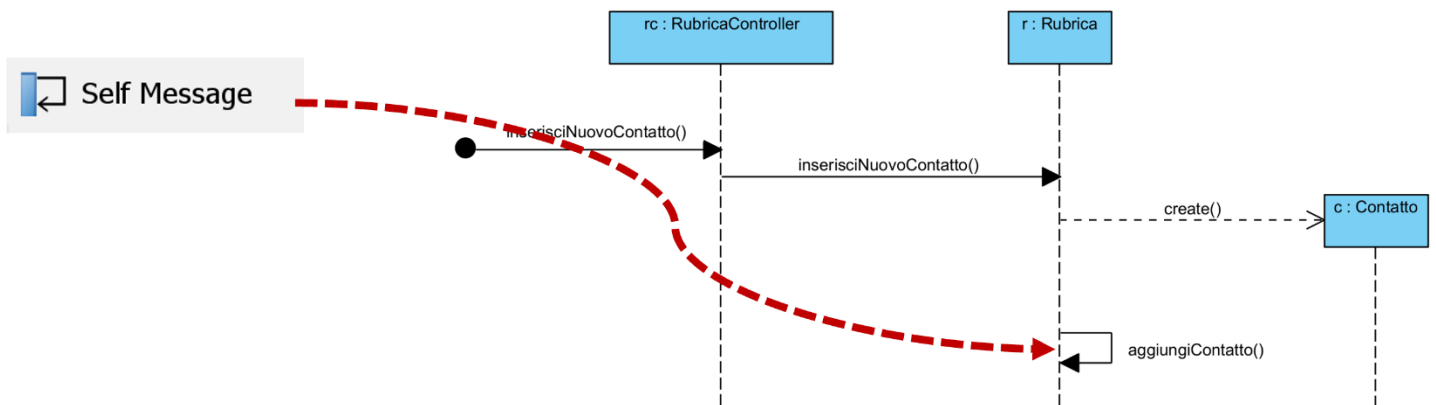
L’operazione provocherà un riposizionamento della lifeline all’altezza del messaggio di creazione, a dimostrazione del fatto che l’istanza “c” di “Contatto” esisterà dal momento in cui verrà effettivamente creata.



Non è necessario, ma per fini didattici aggiungiamo, a conclusione del diagramma di sequenza, anche un messaggio di tipo **self** con origine e destinazione la lifeline “Rubrica”, avente lo scopo di aggiungere alla propria collezione di contatti il nuovo contatto appena creato. Come prima cosa, tornando sul diagramma delle classi SW, aggiungiamo l’operazione aggiungiContatto() alla classe “Rubrica”.

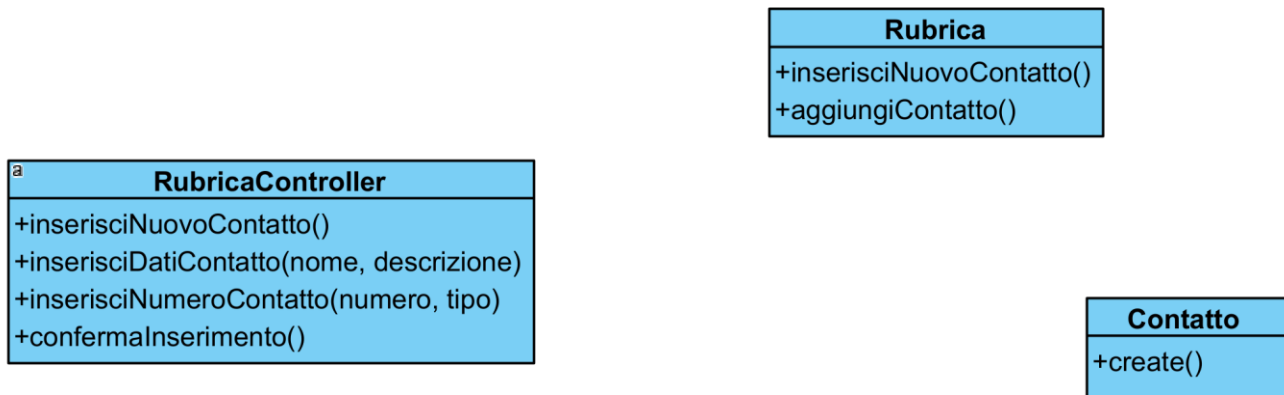


Tornando ora nel diagramma di sequenza, dalla **Palette** selezioniamo **Self Message** e, senza trascinare ma con un solo click del tasto sinistro del mouse, clicchiamo sopra la linea tratteggiata della lifeline “Rubrica”, subito dopo il messaggio create(). Come al solito, verrà richiesto di selezionare un’operazione, quindi indichiamo aggiungiContatto(). Il risultato è il seguente.



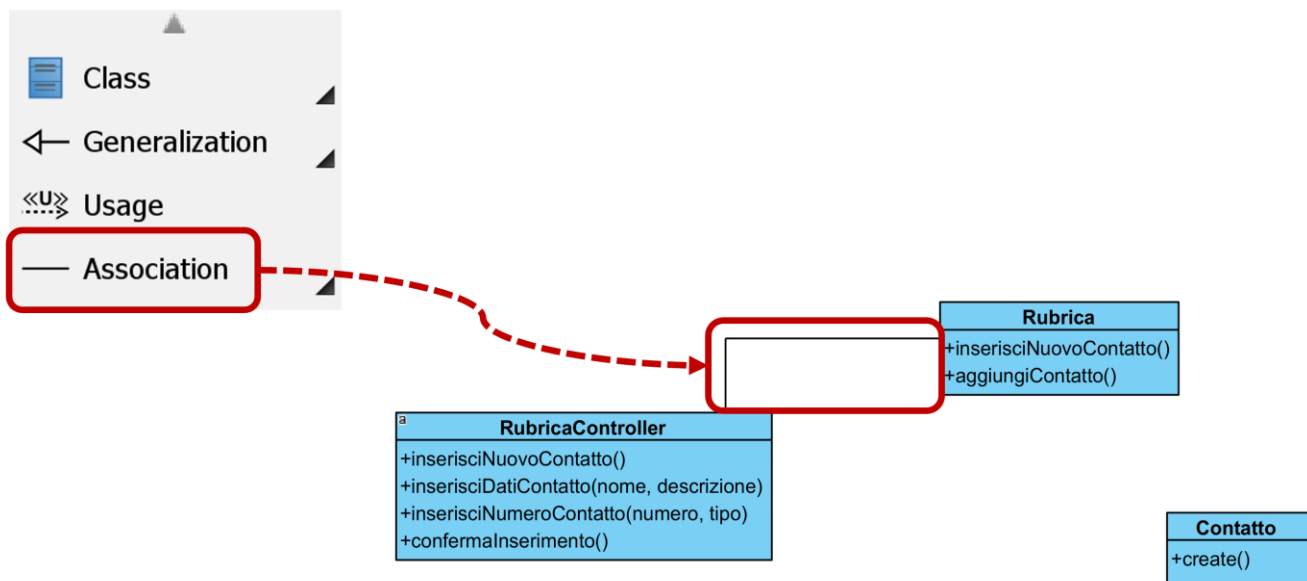
Completamento Diagramma delle classi SW con associazioni, parametri e attributi

Torniamo ora sul diagramma delle classi SW.



Osservando le post-condizioni del contratto **inserisciNuovoContatto** troviamo "l'istanza *c* di Contatto è stata associata alla Rubrica *r*". Dobbiamo quindi completare il diagramma delle classi aggiungendo le associazioni.

Dalla **Palette** a sinistra selezioniamo la voce **Association**, e trasciniamo da "RubricaController" a "Rubrica", in modo da creare un'associazione tra le due classi.



Clicchiamo ora col tasto destro del mouse sull'associazione appena creata e selezioniamo **Open Specification**. Quindi, dalla finestra che si apre, selezioniamo `<Unspecified>` per la voce **Navigable** associata alla classe "RubricaController" e `True` per la stessa voce associata a "Rubrica", e confermiamo.

Association Specification

×

General

▼

Name:

Visibility:

<Unspecified>

▼

Association End From

Role:

...

Element:

RubricaController

...

Multiplicit...

<Unspecified>

▼

Navigable:

<Unspecified>

▼

Association End To

Role:

...

Element:

Rubrica

...

Multiplicit...

<Unspecified>

▼

Navigable:

True

▼

Description:

B

▼

≡

▼

≡

▼

F

▼

▼

▼

▼

+

▼

▼

▼

▼

▼

☐ Abstract

☐ Leaf

☐ Derived

☐ Final specialization

Reset

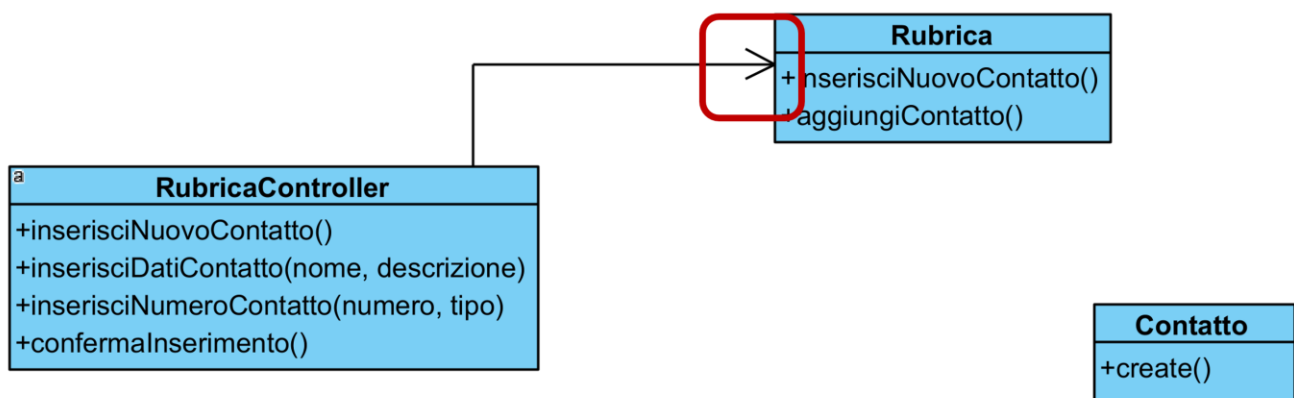
OK

Cancel

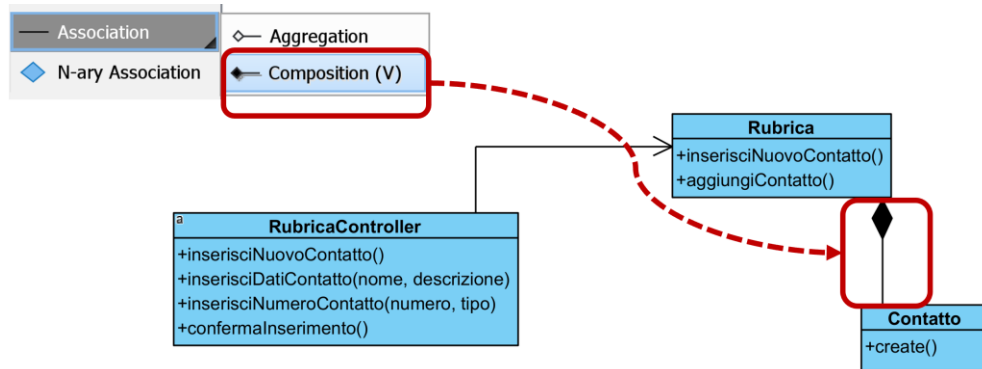
Apply

Help

Questo ci permetterà di visualizzare la navigabilità dell'associazione, che da "RubricaController" va verso "Rubrica".



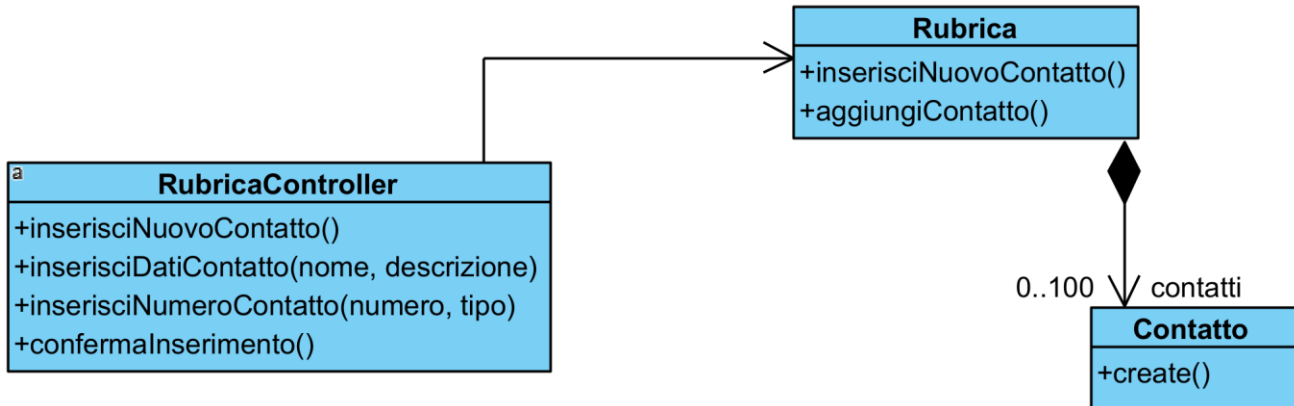
Allo stesso modo, dobbiamo aggiungere l'associazione tra "Rubrica" e "Contatto". In questo caso non si tratta però di una semplice associazione: una rubrica è composta da molti contatti e un contatto appartiene a un'unica rubrica – e non può esistere in assenza di essa. Abbiamo quindi bisogno di una **composizione** (diamante nero). Dalla **Palette** clicchiamo sul triangolino nero di fianco alla voce **Association**, per espandere le opzioni, e selezioniamo **Composition**, trascinando da "Rubrica" a "Contatto".



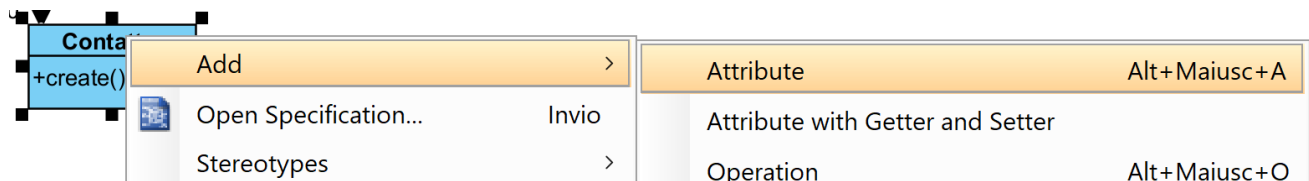
Ricordandoci che una rubrica può avere da 0 a 100 contatti, selezioniamo la composizione appena aggiunta nel diagramma e, sempre dalla finestra aperta via tasto destro > **Open Specification**: modifichiamo la voce **Multiplicity** dal lato "Contatto" inserendo "0..100", selezioniamo <Unspecified> per la voce **Navigable** associata a "Rubrica", e inseriamo "contatti" nel campo **Role** associato a "Contatto".

The screenshot shows the **Association Specification** dialog box. The **General** tab is active. The **Association End From** section is configured with **Role**: (empty), **Element**: **Rubrica**, **Multiplicity**: <Unspecified>, **Navigable**: <Unspecified>, and **Visible**: checked. The **Association End To** section is configured with **Role**: **contatti**, **Element**: **Contatto**, **Multiplicity**: **0..100**, **Navigable**: **True**, and **Visible**: checked. The **Description** field is empty. At the bottom, there are checkboxes for **Abstract**, **Leaf**, **Derived**, and **Final specialization**, all of which are unchecked. The **Reset**, **OK**, **Cancel**, **Apply**, and **Help** buttons are at the bottom.

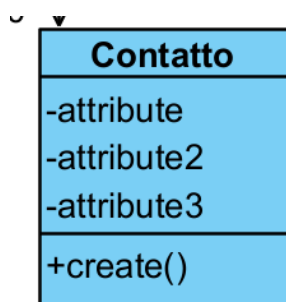
Il risultato sarà il seguente. Si noti che le molteplicità di tipo 1 si possono omettere, in quanto implicite.



Dobbiamo ancora aggiungere gli attributi alle classi. Possiamo prendere come base di partenza il modello di dominio. L'unica classe che presenta attributi è "Contatto", con gli attributi "nome", "descrizione", e "protetto" (le informazioni sul numero di telefono emergeranno quando si andrà a modellare la relativa operazione di sistema e classe SW). Come abbiamo già visto la volta scorsa, per aggiungere attributi a classi ci sono vari modi. Uno di essi è cliccare col tasto destro del mouse sulla classe "Contatto", quindi selezionare **Add > Attribute**.



Eseguiamo l'operazione tre volte per creare tre attributi.



Dato che siamo a livello SW, oltre al nome degli attributi, dovremo considerare altre informazioni per essi, quali **visibilità**, **tipo**, **molteplicità** e **valore predefinito**. Clicchiamo sul primo attributo e, aprendo il menu col tasto destro, selezioniamo la voce **Open Specification**.

Attribute Specification

General

Name:

Classifier: ...

Initial value:

Multiplicity: ☐ Ordered ☒ Unique

Visibility:

Type: ...

Type modifier:

Scope:

Aggregation:

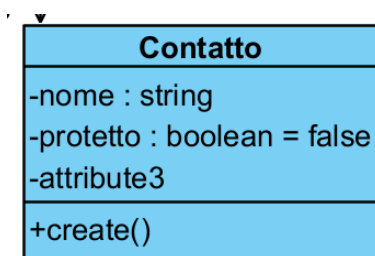
Description:

☐ Derived ☐ Derived union ☐ Setter ☐ Getter

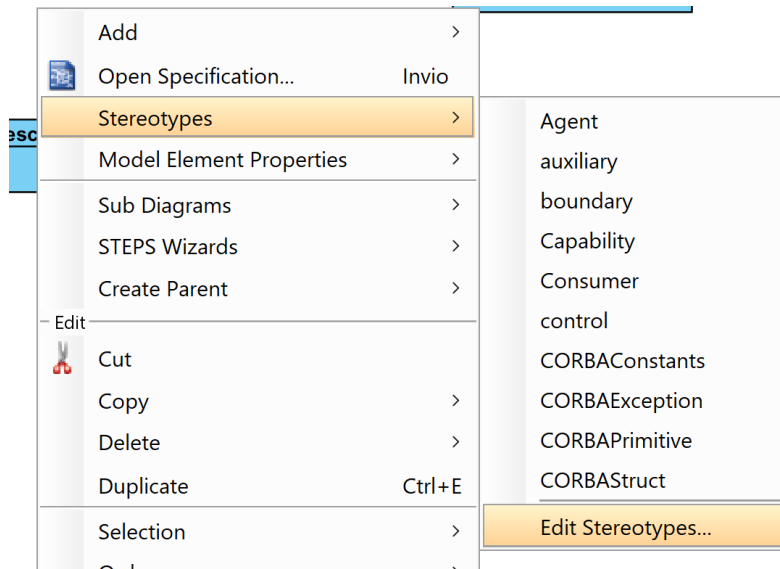
☐ Abstract ☐ Leaf ☐ Read only ☐ ID

Reset OK Cancel Apply Help

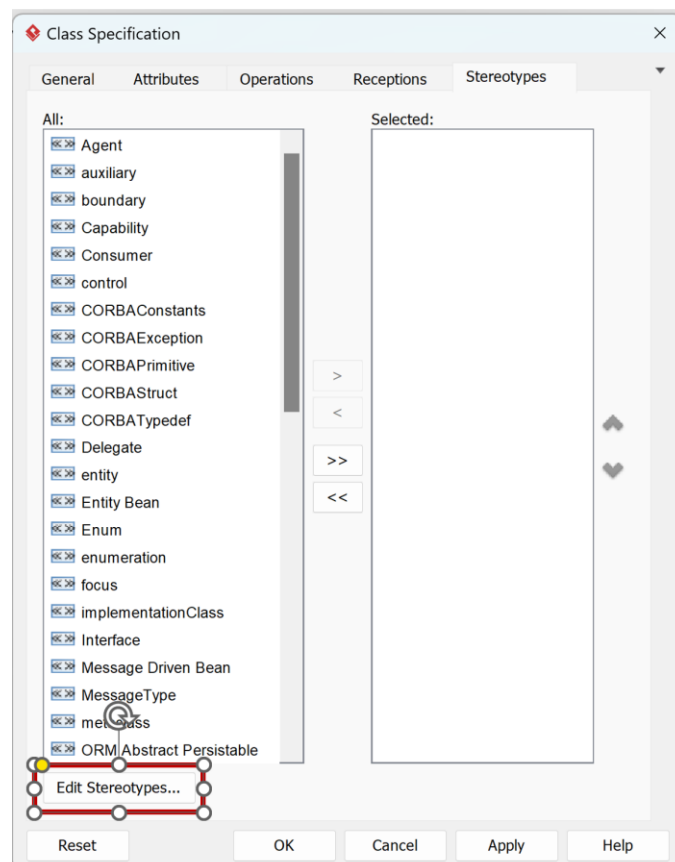
Per quanto riguarda il campo **Name** inseriamo la stringa “nome”, mentre per il campo **Type** selezioniamo il tipo “string” dal menu. Non ci serve invece modificare la molteplicità (**Multiplicity**, di default impostata a 1), la visibilità (**Visibility**, di default impostata su private per gli attributi) o il valore predefinito dell’attributo (**Initial value**, di default vuoto). Ripetiamo l’operazione per il secondo attributo “protetto”: come **Name** inseriamo “protetto”, come **Type** indichiamo boolean, e come **Initial value** inseriamo il valore “false”.



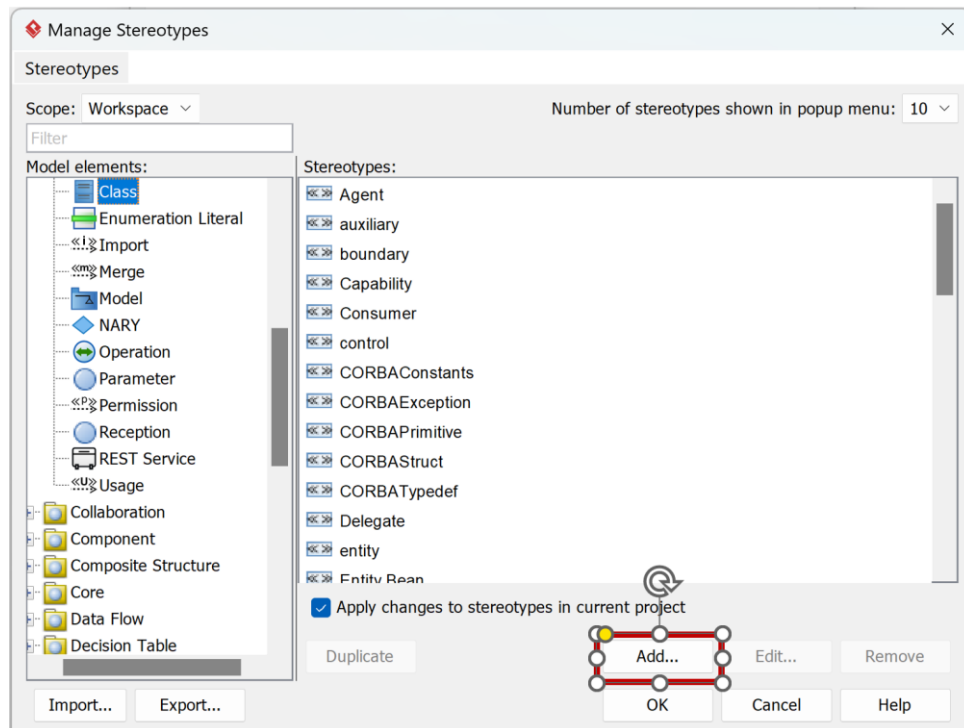
Rimane da gestire l'attributo "descrizione". Ai fini didattici, assumiamo che la descrizione di un contatto non sia una semplice stringa, ma qualcosa di più complesso: introduciamo un **tipo di dato** (anche chiamato **datatype**) mediante *stereotipi* (un concetto UML usato per personalizzare un modello). Dalla **Palette** a sinistra trasciniamo una nuova classe nel diagramma e diamole il nome "Descrizione". Quindi, cliccando col tasto destro del mouse sulla classe, selezioniamo **Stereotypes > Edit Stereotypes**.



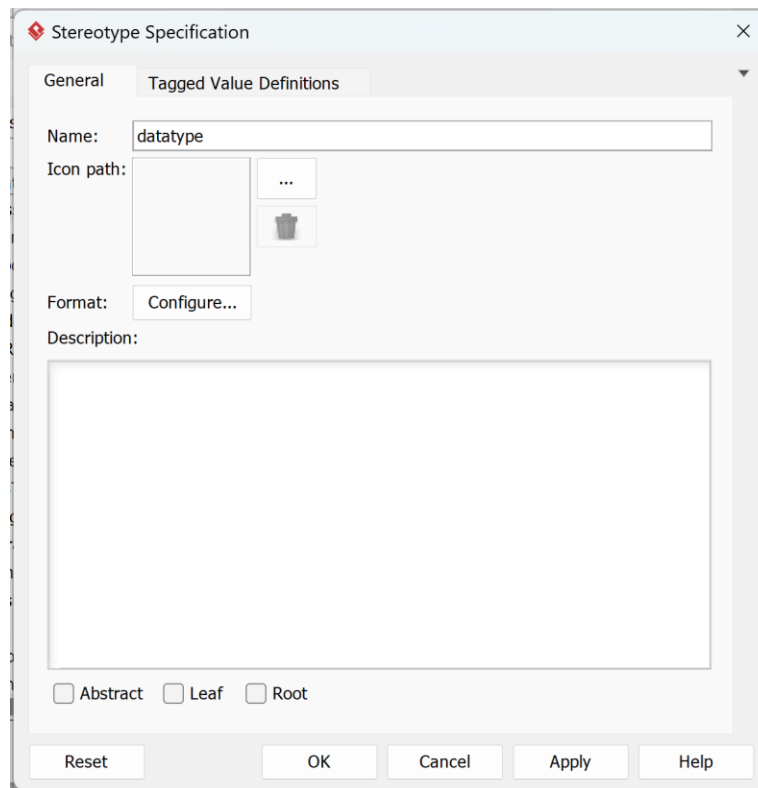
Dalla finestra che si apre, clicchiamo sul bottone **Edit Stereotypes**.



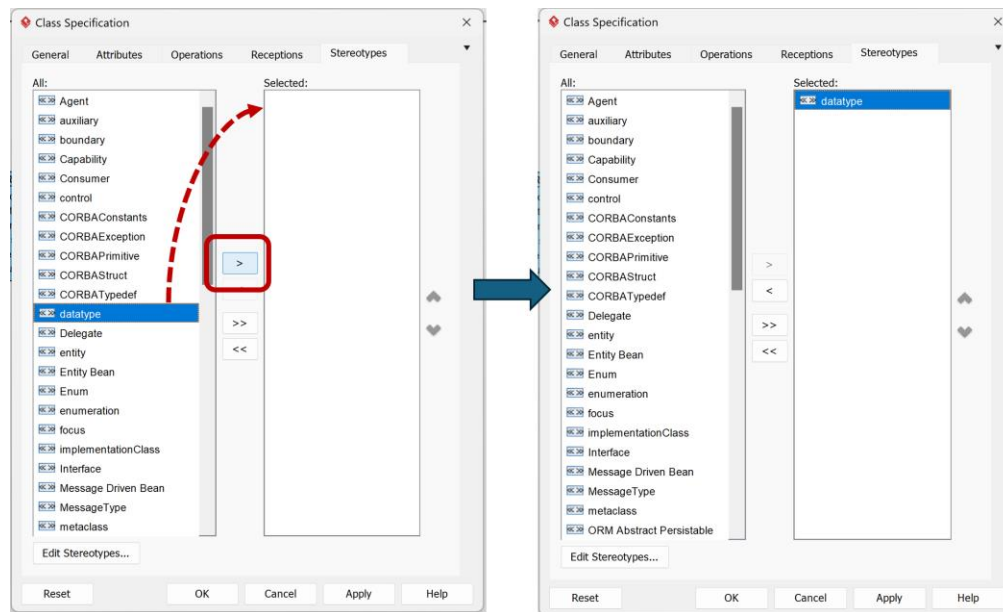
Nella nuova finestra, clicchiamo **Add**.



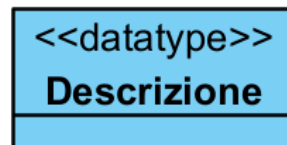
Nella nuova finestra, digitiamo “datatype” nel campo **Name** e confermiamo.



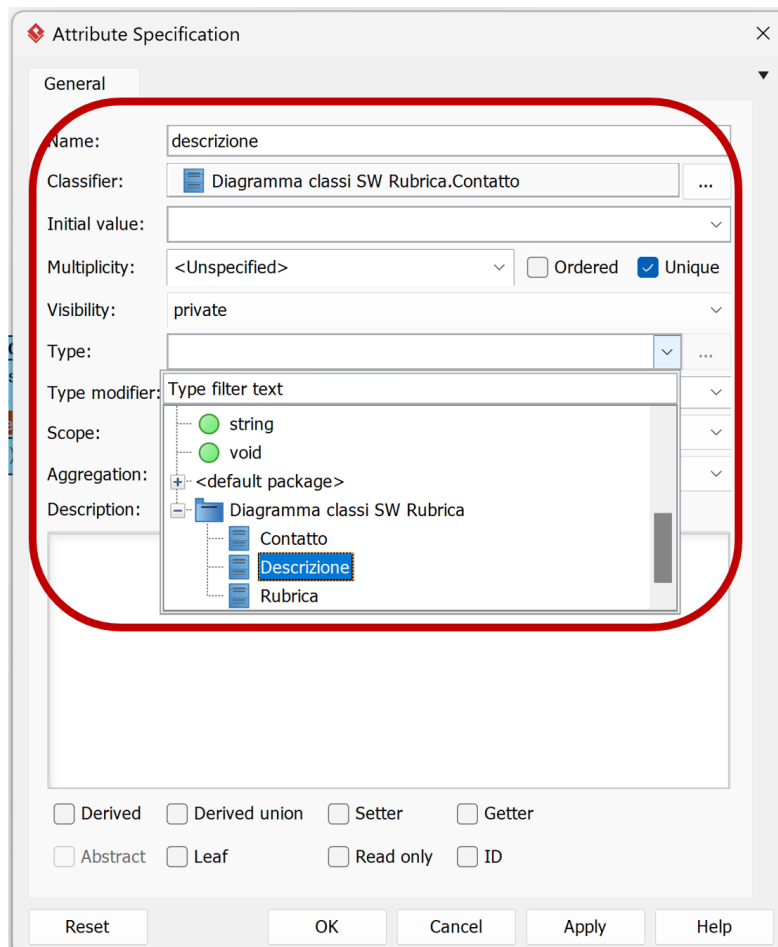
Tornando nella finestra che elenca gli stereotipi, selezioniamo la voce **datatype** appena aggiunta nella lista a sinistra, clicchiamo su > per spostarla nella lista a destra, quindi confermiamo.



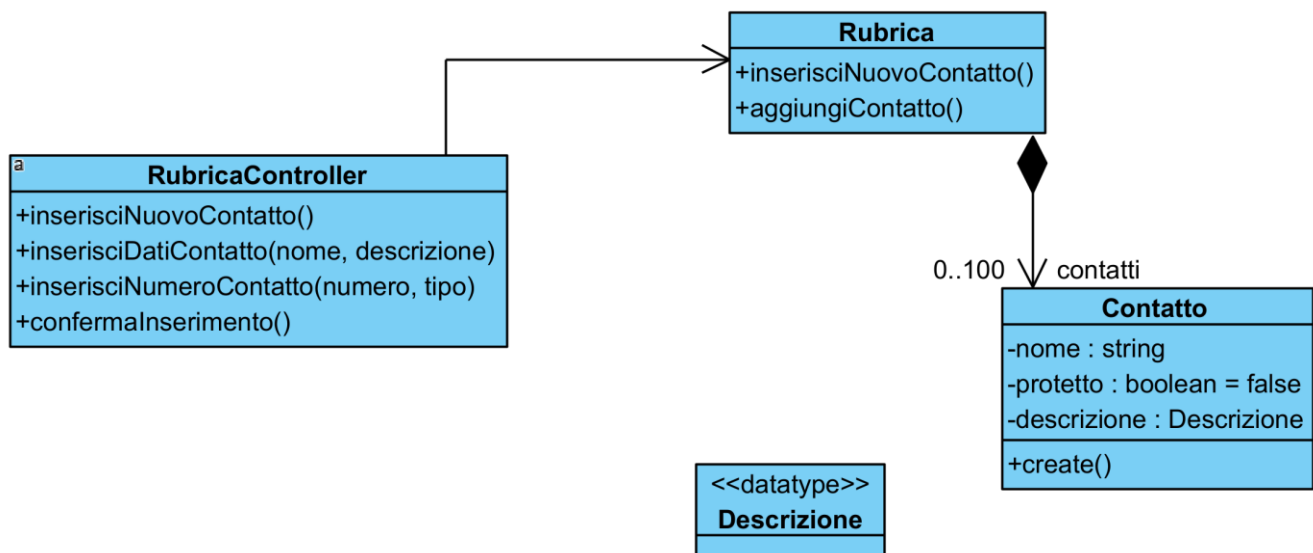
L'operazione aggiungerà uno stereotipo <<datatype>> associato alla classe Descrizione, la cui struttura non è stata definita, ma che sarà utile per identificare i tipi di dato non primitivi.



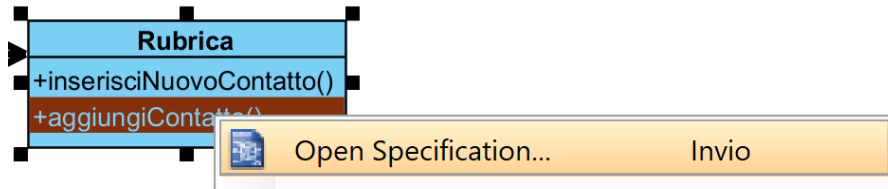
A questo punto, cliccando col tasto destro sull'ultimo attributo della classe "Contatto", selezioniamo **Open Specification**, e dalla finestra che si apre inseriamo "descrizione" nel campo **Name** ed espandiamo le voci sotto **Type** fino a individuare e selezionare il datatype "Descrizione" introdotto.



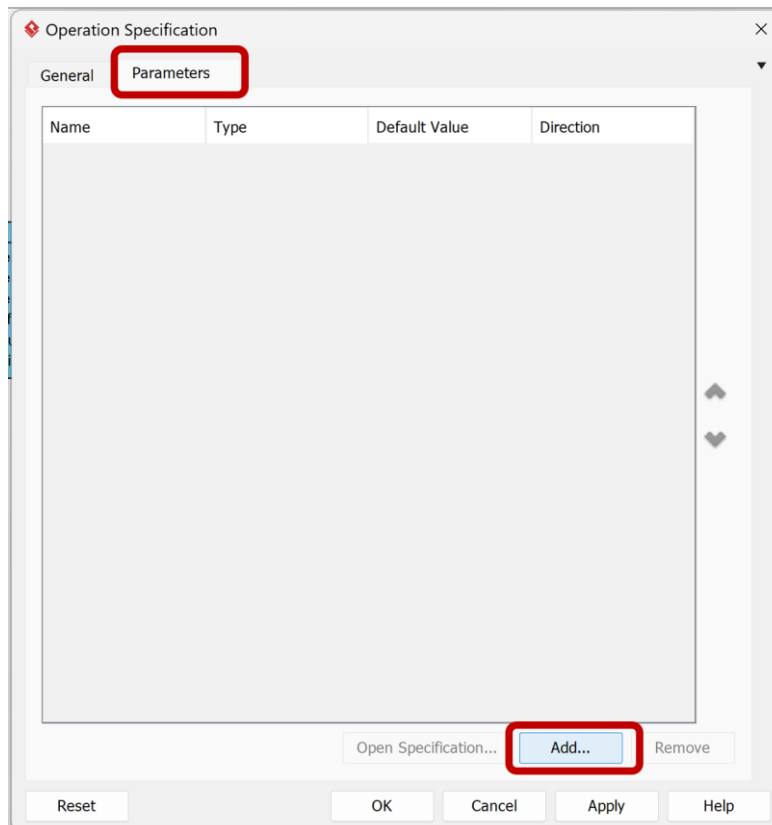
A questo punto il diagramma delle classi SW è il seguente.



Rimane da definire un parametro per l'operazione `aggiungiContatto()` di "Rubrica". Clicchiamo col tasto destro su `aggiungiContatto()`, quindi **Open Specification**.



Dalla finestra che si apre, selezioniamo la scheda **Parameters**, quindi clicchiamo su **Add**.



Questo passaggio inserirà un parametro all'operazione `aggiungiContatto()`. Ragionando su quanto già modellato, noi vogliamo un parametro "c" di tipo "Contatto", in quanto si tratta del contatto creato che vogliamo aggiungere all'insieme di contatti appartenenti alla rubrica. Al parametro appena creato quindi assegniamo il nome "c" modificando il campo **Name**, clicchiamo nel campo **Type** e scorriamo fino a individuare la voce "Contatto", e confermiamo.

Parameter Specification

General

Name:

Operation:

Type:

Type modifier:

Direction:

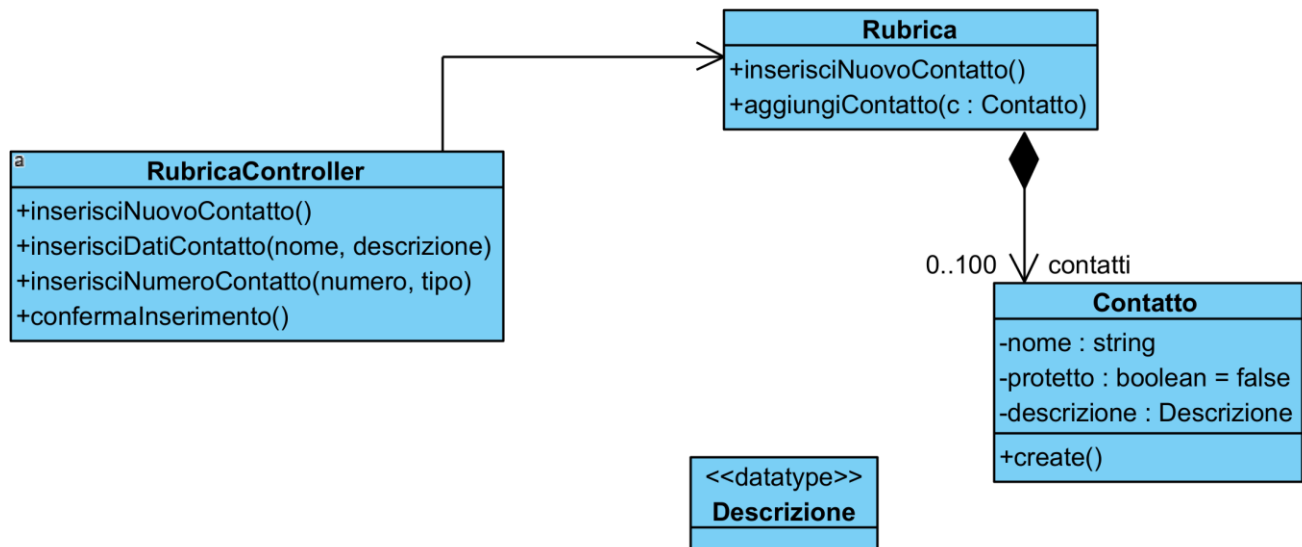
Default value:

Multiplicity: ☐ Ordered ☒ Unique

Description:

Reset OK Cancel Apply Help

Il risultato finale è il seguente.



Per concludere, torniamo sul diagramma di sequenza e osserviamo come il messaggio **self** di Rubrica `aggiungiContatto()` sia stato aggiornato con il parametro appena aggiunto.

